

Introducción



- Lenguaje de programación de alto nivel.
- Multiplataforma
- Multiparadigma
 - Imperativo y/o Procedural
 - Orientado a objetos
 - Funcional (limitado)
- Interpretado
 - Precompilado (.pyc)
 - Modo interactivo: Interprete de comandos

✓ Conceptos básicos

Bloques de código

- Indentación obligatoria
 - No existen las llaves { } para delimitar los bloques de código. Se utiliza : despues de la cabecera del bloque de código.
 - Los bloques de código son delimitados mediante indentación.
- No hay puntos y coma ; al final de las líneas de código.

```
# Probaremos hacer el factorial de 5
x = 5

def factorial(x):
    if x == 0:
        return 1
    else:
        return x * factorial(x - 1)

x = factorial(x)

# Imprimimos el factorial
print(x)
```

120

```
# TODO
# Cual es el factorial de 4
factorial(4)
```

24

✓ Tipado dinámico

El tipo de las variables es definido dinámicamente.

```
var = 12
type(var)
```

int

```
var + 1
```

```
13
```

```
var = 12  
type(var)
```

```
int
```

```
var + 1
```

```
13
```

✓ Tipos de datos

Booleano

```
True  
False
```

Cadena

```
# cadena  
'cadena'(en py3, todos son unicode)  
# unicode  
u'cadena' (solo necesario para py 2)
```

Número entero

```
23
```

✓ Número real

```
# coma flotante: tipo float  
3.14  
# coma flotante de mayor precisión  
Decimal(3.14)
```

```
0.1+0.2
```

```
0.30000000000000004
```

```
from decimal import Decimal  
  
float(Decimal("0.1") + Decimal("0.2"))
```

```
0.3
```

```
import math  
math.pi
```

```
3.141592653589793
```

```
# coma flotante  
from decimal import Decimal  
Decimal(math.pi)
```

```
Decimal('3.141592653589793115997963468544185161590576171875')
```

✓ Número complejo

```
5.4+2j
```

```
z = 5.4 + 2j
print(z.real)
print(z.imag)
```

```
5.4
2.0
```

✓ Estructuras de datos

✓ Lista (lista mutable)

```
lista = [21, 'cadena', True]
```

✓ Operadores

```
lista[i]
lista[i:j]
lista[i:j:k]
lista.append(x)
lista.extend(lista)
lista.insert(i,x)
lista.remove(x)
lista.pop(i) # i es opcional
lista.index(x)
lista.count(x)
lista.sort()
lista.reverse()
```

```
# TODO: Prueba estos operadores e indica su función
lista = [21, 'cadena', True, 'amigo']
print(lista, "\n")

# lista[i]: recupera el valor de la posicion i de la lista
lista = [21, "cadena", True]
print("lista[2] da este resultado: " + lista[1])

# lista[i:j]: recupera los valores desde i hasta j - 1
lista = [21, "cadena", True]
print(lista[1:3])

# lista[i:j:k]: recupera los valores desde i hasta j-1 con saltos de k
lista = [21, "cadena", True]
print(lista[0:3:2])

# lista.append(x): añade un valor al final de la lista
lista = [21, "cadena", True]
lista.append(57)
print(lista)

#lista.extend(lista): añade una lista al final
lista = [21, "cadena", True]
lista.extend(lista)
print(lista)

#lista.insert(i,x): añade un valor en la posicion i
lista = [21, "cadena", True]
lista.insert(1,47)
print(lista)

#lista.remove(x): elimina la primera ocurrencia del valor de la lista
lista = [21, "cadena", True]
lista.remove(21)
print(lista)
```

```
[21, 'cadena', True, 'amigo']

lista[2] da este resultado: cadena
['cadena', True]
[21, True]
[21, 'cadena', True, 57]
[21, 'cadena', True, 21, 'cadena', True]
[21, 47, 'cadena', True]
['cadena', True]
```

```
#lista.pop(x): elimina y muestra la primera ocurrencia del valor de la lista
lista = [21, "cadena", 5]
lista.pop(2)
# i es opcional
print(lista)
```

```
[21, 'cadena']
```

```
# TODO Crear una pila con una lista
lista = [21, "cadena", 5]
lista.append('primero')
print(lista)

lista.pop()
print(lista)
```

```
[21, 'cadena', 5, 'primero']
[21, 'cadena', 5]
```

```
# TODO Crear una cola con una lista
lista = [21, "cadena", 5]
lista.append('ultimo')
print(lista)
lista.pop(0)
print(lista)
```

```
[21, 'cadena', 5, 'ultimo']
['cadena', 5, 'ultimo']
```

Tupla (lista no mutable)

```
(21, 'cadena', True)
```

✓ Conjunto (mutable)

```
conjunto = set([21, 'cadena', True, 'cadena'])
```

```
conjunto
```

```
{21, True, 'cadena'}
```

✓ Operadores

```
conjunto.add(x)
conjunto.remove(x) # Lanza excepción
conjunto.discard(x) # No lanza excepción
conjunto.pop()
conjunto.update(conjunto)
x in conjunto
conjunto1 <= conjunto2
conjunto1 >= conjunto2
conjunto1 - conjunto2
conjunto1 | conjunto2
conjunto1 & conjunto2
conjunto1 ^ conjunto2
```

```
# TODO Prueba estos operadores e indica su función
conjunto = set([21, 'cadena', True, 'cadena'])
print('Conjunto original:\n', conjunto, '\n\n')
```

```
# conjunto.add(amigo)
print("conjunto.add(x)::")
conjunto = set([21, 'cadena', True, 'cadena'])
conjunto.add('amigo')
print(conjunto, '\n')
```

```
# conjunto.remove(21) # Lanza excepción
```

```

print("conjunto.remove(x):")
conjunto = set([21, 'cadena', True, 'cadena'])
conjunto.remove(21)
print(conjunto, '\n')

# conjunto.discard(x) # No lanza excepción
print("conjunto.discard(x):")
conjunto = set([21, 'cadena', True, 'cadena'])
conjunto.discard(x)
print(conjunto, '\n')

# conjunto.pop()
print("conjunto.pop():")
conjunto = set([21, 'cadena', True, 'cadena'])
conjunto.pop()
print(conjunto, '\n')

# conjunto.update(conjunto)
print("conjunto.update(conjunto):")
conjunto = set([21, 'cadena', True, 'cadena'])
conjunto.update(conjunto)
print(conjunto)

```

```

Conjunto original:
{True, 'cadena', 21}

conjunto.add(x)::
{True, 'cadena', 'amigo', 21}

conjunto.remove(x):
{True, 'cadena'}

conjunto.discard(x):
{True, 'cadena', 21}

conjunto.pop():
{'cadena', 21}

conjunto.update(conjunto):
{True, 'cadena', 21}

```

```

# x in conjunto
print("x in conjunto")
conjunto = set([21, 'cadena', True, 'cadena'])
print(21 in conjunto, '\n')

# conjunto1 <= conjunto2
print("conjunto1 <= conjunto2")
conjunto1 = set([1, 2, 3])
conjunto2 = set([1, 2, 3, 4])
print(conjunto1 <= conjunto2, '\n')

# conjunto1 >= conjunto2
print("conjunto1 >= conjunto2")
conjunto1 = set([1, 2, 3])
conjunto2 = set([1, 2, 3, 4])
print(conjunto1 >= conjunto2, '\n')

# conjunto1 - conjunto2
print("conjunto1 - conjunto2")
conjunto1 = set([1, 2, 3])
conjunto2 = set([1, 2, 4])
print(conjunto1 - conjunto2, '\n')

# conjunto1 | conjunto2
print("conjunto1 | conjunto2")
conjunto1 = set([1, 2, 3, 6])
conjunto2 = set([1, 2, 3, 4])
print(conjunto1 | conjunto2, '\n')

# conjunto1 & conjunto2
print("conjunto1 & conjunto2")
conjunto1 = set([1, 2, 3])
conjunto2 = set([1, 2, 3, 4])
print(conjunto1 & conjunto2, '\n')

# conjunto1 ^ conjunto2
print("conjunto1 ^ conjunto2")
conjunto1 = set([1, 2, 3])
conjunto2 = set([1, 2, 3, 4])
print(conjunto1 ^ conjunto2)

```

```
x in conjunto
True

conjunto1 <= conjunto2
True

conjunto1 >= conjunto2
False

conjunto1 - conjunto2
{3}

conjunto1 | conjunto2
{1, 2, 3, 4, 6}

conjunto1 & conjunto2
{1, 2, 3}

conjunto1 ^ conjunto2
{4}
```

Conjunto (no mutable)

```
frozenset([21, 'cadena', True])
```

▼ Diccionario

```
diccionario = {'k1': 1, 'k2': 2}
diccionario = dict(k1 = 1, k2 = 2)
```

▼ Operadores

```
diccionario[clave] # Lanza excepción
diccionario.get(clave) # No lanza excepción
diccionario[clave] = valor
clave in diccionario
diccionario.keys()
diccionario.items()
```

```
diccionario = {'k1': 1, 'k2': 2, 'k3':3}
print(diccionario, "\n")

# TODO Prueba estos operadores e indica su función

# diccionario[clave]      # Accede al valor de 'clave'. Si no existe, lanza excepción KeyError.
print("diccionario['k1']  : ", diccionario['k1'] )

# diccionario.get(clave)  # Accede al valor de 'clave'. Si no existe, devuelve None (o un valor por defecto si lo
print("diccionario.get('k2'): ", diccionario.get('k2'))

# diccionario[clave] = val # Inserta o actualiza la clave con el valor 'val'.
diccionario['k3'] = 4

# clave in diccionario    # Devuelve True si la clave existe en el diccionario, False si no.
print("'k2' in diccionario : ", 'k2' in diccionario)

# diccionario.keys()      # Devuelve una vista con todas las claves del diccionario.
print("diccionario.keys() : ", diccionario.keys())

# diccionario.items()     # Devuelve una vista con pares (clave, valor).
print("diccionario.items() : ", diccionario.items())

{'k1': 1, 'k2': 2, 'k3': 3}

diccionario['k1']      : 1
diccionario.get('k2'): 2
'k2' in diccionario   : True
diccionario.keys()    : dict_keys(['k1', 'k2', 'k3'])
diccionario.items()   : dict_items([('k1', 1), ('k2', 2), ('k3', 4)])
```

▼ Métodos comunes

Borrar variables o elementos de una estructura de datos

```
del variable
del list[i]
del list[i:k]
```

```
# TODO Borra los dos últimos elementos de la lista creada anteriormente
lista = [1, 2, 3, 4, 5, 6]
print(lista)
```

```
del lista[len(lista) - 2:len(lista)]
print(lista)
```

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4]
```

✓ Longitud de una cadena o estructura de datos – len(cadena)

```
len(lista)
len(cadena)
```

Iterador de una estructura de datos

```
iterador = iter(lista)
iterador = iter(mapa)
next(iterador) # Lanza excepción al terminar
```

```
# Ejemplo uso iterador
lista = [10, 20, 30]
iterador = iter(lista)
print(next(iterador)) # Imprime 10
print(next(iterador)) # Imprime 20
print(next(iterador)) # Imprime 30
#print(next(iterador)) # Lanza StopIteration
```

```
# Evitamos la excepción
try:
    while True:
        elemento = next(iterador)
        print(elemento)
except StopIteration:
    print("Fin de la iteración")
```

```
10
20
30
Fin de la iteración
```

✓ Condiciones y bucles

Condiciones

```
and, or, not
<, >, ==, <=, >=
in, not in
is, is not # Objetos mutables
```

```
# TODO Crear dos variables que cumplan la condición ==
# y no cumplan la condición is
```

```
x = [1, 2]
y = [1, 2]
```

```
print(x == y)
print(x is y)
```

```
True
False
```

```
### If else
```

```
var = 0
if var:
    print("True")
elif not var:
    print("False")
else:
    print("Se ha roto el mundo.")
```

```
False
```

```
var0 = None # Null

if var0 is None: # Comprueba si está inicializada
    var0 = 1 # Inicializamos

var0+=1
print(var0)
```

```
2
```

¿Qué pasa si está inicializado a cero o False? ¿La comprobación funcionaría?

```
#var0 = None # Null
var0 = 0
#var0 = False
# not var0 es cierto para 0, None y False)
if not var0 : # No comprobar si una variable está inicializada con este método
    var0 = 1 # Inicializamos

var0+=1
print(var0)
```

```
2
```

While

```
while condición:
    código
```

```
#TODO Crear una cuenta atras con el bucle while
i = 5
while(i >= 0):
    print(i)
    i = i - 1
```

```
5
4
3
2
1
0
```

For

```
for x in lista_set:
    código
```

Itera sobre una copia superficial (util si voy a cambiar la lista dentro del bucle)

```
for x in lista[:]:
    lista.pop() # modifica la lista original
```

```
for k,v in diccionario.items(): # Itera sobre las pares clave valor de un diccionario
    código
```

```
#TODO Mostrar las claves del diccionario creado anteriormente junto a su valor más uno
```



```
diccionario = {'k1': 1, 'k2': 2}

for x in diccionario:
    print(x, diccionario[x] + 1)
```

```
k1 2
k2 3
```

✓ Range

```
for i in range(longitud):
    código

for i in range(inicio, fin):
    código

for i in range(inicio, fin, incremento):
    código
```

#TODO Crear una cuenta atras con el bucle for y haciendo uso de range

```
for x in range(6, 0, -1):
    print(x)
```

```
6
5
4
3
2
1
```

Break, continue, else, pass

```
while condición:
    if condición:
        continue # Salta una iteración

while condición:
    if condición:
        break # Finaliza un bucle while o for

while condición:
    código
else: # Se ejecuta cuando la condición es false, con break no se ejecuta
    código

while True:
    pass # No hace nada
```

✓ Conceptos no tan básicos

Programación funcional

✓ Función anonima lambda

```
lambda argumentos: codigo
```

```
f = lambda x,y,z: x*y*z

f(4,4,5)
```

```
80
```

✓ map

```
map(función, secuencia): aplica la funcion a cada elemento de una secuencia
```

```
def cuadrado(x): return x**2
#Hay que hacer un casting a list para poder ver el contenido
list(map(cuadrado, range(4)))
```

```
[0, 1, 4, 9]
```

```
#TODO definir la función cuadrado dentro de la función map (en línea) usando la función anonima lambda
```

```
list(map(lambda x: x**2, [1, 2, 3, 4]))
```

```
[1, 4, 9, 16]
```

```
#TODO Crear una nueva lista con la suma del valor de los elementos
# con el mismo indice de las lista l1 y l2 utilizando map
```

```
l1 = range(5)
l2 = range(4, -1, -1)
```

▼ filter

```
filter(función, secuencia): filtra los valores V (segun una funcion booleana) de la secuencia
```

```
es_par = lambda x: not x%2
list(filter(es_par, range(10)))
```

```
[0, 2, 4, 6, 8]
```

▼ reduce

```
functools.reduce(función, secuencia): : aplica la función function de dos argumentos acumulativamente a los elementos
```

```
import functools

functools.reduce(lambda x,y: x*y, range(1, 5))
```

```
24
```

▼ Listas comprendidas

```
[x for x in data_struct]

[x for x in data_struct if condition]

[(x,y) for x in data_struct for y in data_struct]

[[x+y for x in data_struct] for y in data_struct]
```

```
# Lista del 0 al 9. Sí, se puede hacer directamente con range
[x for x in range(10) if x > 5]
```

```
[6, 7, 8, 9]
```

```
# Equivalente a este código
lista = []
for x in range(10):
    lista.append(x)
lista
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Combinatoria posible para dos numeros 3 números binarios
binario = [0,1]
[(x,y,z) for x in binario for y in binario for z in binario]
```

```
[(0, 0, 0),
 (0, 0, 1),
 (0, 1, 0),
 (0, 1, 1),
 (1, 0, 0),
 (1, 0, 1),
 (1, 1, 0),
 (1, 1, 1)]
```

```
# TODO: ¿A qué código equivale?
```

```
binario = [1, 0]
combinaciones = []

for x in binario:
    for y in binario:
        for z in binario:
            combinaciones.append((x, y, z))

print(combinaciones)
```

```
[(1, 1, 1), (1, 1, 0), (1, 0, 1), (1, 0, 0), (0, 1, 1), (0, 1, 0), (0, 0, 1), (0, 0, 0)]
```

```
# Matriz de distancias Manhattan a la coordenada (0,0)
[[x+y for x in range(5)] for y in range(5)]
```

```
[[0, 1, 2, 3, 4],
 [1, 2, 3, 4, 5],
 [2, 3, 4, 5, 6],
 [3, 4, 5, 6, 7],
 [4, 5, 6, 7, 8]]
```

```
# TODO: ¿A qué código equivale?
```

```
lista_principal = []

for x in range(5):
    lista = []
    for y in range(5):
        lista.append(x + y)
    lista_principal.append(lista)

print(lista_principal)
```

```
[[0, 1, 2, 3, 4], [1, 2, 3, 4, 5], [2, 3, 4, 5, 6], [3, 4, 5, 6, 7], [4, 5, 6, 7, 8]]
```

▼ map

```
# TODO Crear mediante listas comprendidas una lista
# con los cuadrados de los números del 0 al 4
```

```
# def cuadrado(x): return x**2

cuadrado: lambda x: x**2

[cuadrado(x) for x in range(5)]
```

```
[0, 1, 4, 9, 16]
```

▼ filter

```
# TODO Crear mediante listas comprendidas una lista
# con los números pares de los números del 0 al 10
pares = [x for x in range(0, 11) if x % 2 == 0]
print(pares)
```

```
[0, 2, 4, 6, 8, 10]
```

```
#Lista con un booleano indicando si los números del uno al 10 son pares o no
print(list(range(1, 11)))
[True if not x%2 else False for x in range(1, 11)]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[False, True, False, True, False, True, False, True, False, True]
```

✓ Funciones

Definición

```
def funcion(argumentos):
    codigo

f = funcion
f(argumentos)
```

Parametros por defecto

```
def funcion(arg1, arg2=val2): # Valor por defecto val2 para arg2
    Codigo

funcion(val1)
funcion(val1, arg2)
funcion(val1, arg2=val2)
funcion(arg2=val2, arg1=val1)
```

✓ Argumentos indefinidos

Lista de argumentos sin nombre

```
def funcion(*lista):
    for argumento in lista:
        codigo
```

```
def suma(*numeros):
    import functools
    return functools.reduce(lambda x,y: x+y, numeros)
# numeros es una tupla/lista de longitud variable
print(suma(1,2))
print(suma(1,2,3,4))
```

```
3
10
```

✓ Lista de argumentos con nombre

```
def funcion(arg1, arg2, **diccionario):
    for k,v in diccionario.items():
        codigo

funcion(val1, val2, arg3=val3, arg4=val4)
```

```
# Definir una funcion para configurar perfil de usuario
# No conocemos el numero de propiedades. Se acumulan en un diccionario
# propiedades es un diccionario de longitud variable
def configurar_perfil(usuario, **propiedades):
    print(f"Configurando perfil de usuario {usuario}")
    for propiedad, valor in propiedades.items():
        print(f"{propiedad}: {valor}")
```

```
# Ahora llamo a la función
configurar_perfil("pablo", edad=28, nombre="Pablo", apellido="Ramos")
```

```
Configurando perfil de usuario pablo
edad: 28
nombre: Pablo
apellido: Ramos
```

Desempaquetado de argumentos

Podemos pasar los argumentos como una lista o un diccionario

```
1) Defino la funcion con argumentos
def funcion(arg3, arg4=val4):
    codigo

2) Defino mi diccionario/lista/tupla:
diccionario = {arg3: val3, arg4: val4}

3) Paso mi diccionario/lista/tupla como argumentos:
funcion(**diccionario) # ** desempaqueta el diccionario
funcion(*lista) # desempaqueta la lista/tupla
```

```
# EJEMPLO LISTA
def sumar(a, b, c):
    return a + b + c

argumentos = [1, 2, 3]
resultado = sumar(*argumentos)
print(resultado)
```

6

```
# EJEMPLO TUPLA
def sumar(a, b, c):
    return a * b + c

argumentos = (4, 5, 6)
resultado = sumar(*argumentos)
print(resultado)
```

26

```
#EJEMPLO DICCIONARIO
def configurar_usuario(nombre, edad, email):
    print(f"Usuario configurado con Nombre: {nombre}, Edad: {edad}, Email: {email}")

datos_usuario = {
    'nombre': 'Elena',
    'edad': 34,
    'email': 'elena@example.com'
}

configurar_usuario(**datos_usuario)
```

Usuario configurado con Nombre: Elena, Edad: 34, Email: elena@example.com

Clases

Sintaxis

```
class NombreClase: # Hereda de la clase Object
    codigo
```

Constructor

```
class NombreClase:
    def __init__(self, argumento):
        self.variable = argumento
```

Instancia

```
miClase = NombreClase(argumento)
miClase.variable = nuevoValor
```

```
# TODO ¿Qué diferencia hay entre la
# variable var de la clase1 y la clase 2?
class Clase1:
    var = 0
    def __init__(self):
        pass

class Clase2:
    def __init__(self):
        var = 0

class Clase3:
    def __init__(self):
        self.var = 0
```

✓ Herencia

```
# NombreClase hereda de ClasePadre
class NombreClase(ClasePadre):
    #Constructor de NombreClase
    def __init__(self, argumentos):
        #Constructor ClasePadre
        ClasePadre.__init__(self)
    #Método de NombreClase que sobrescribe el metodo, si existe en ClasePadre
    def metodo(self):
        ClasePadre.metodo(self)

# NombreClase hereda de varias Clases
class NombreClase(ClasePadre1, ClasePadre2,...):
    codigo
```

Variables y métodos privados

No existen en python, no obstante hay una convención que dice que si una variable esta precedida de dos guiones bajos, es una variable privada.

```
class NombreClase:
    def __init__(self, argumentos):
        self.__variablePrivada = argumento
    def setVariablePrivada(self, argumento):
        self.__variablePrivada = argumento
```

Property: Analogo a getter y setter

```
class NombreClase:
    def __init__(self, argumentos):
        self.__variablePrivada = argumento

    @property #decorador getter
    def variablePrivada(self):
        return self.__variablePrivada

    @variablePrivada.setter # decorador setter
    def variablePrivada(self, argumento):
        self.__variablePrivada = argumento
```

```
# TODO Crear una clase cuyo metodo constructor reciba un diccionario
# como argumento e inicialice tantas variables del objeto como
# elementos tenga el diccionario. La clave sera el nombre de la
# variable y el valor su valor.
# self.valor_clave contiene valor del diccionario
```

```
# TODO Crear una clase cuyo metodo constructor reciba un diccionario
# como argumento e inicialice tantas variables del objeto como
# elementos tenga el diccionario. La clave sera el nombre de la
# variable y el valor su valor.
# self.valor_clave contiene valor del diccionario
```

```
# SOLUCIÓN
class Profesor(object):
    def __init__(self, **dict):
        self.__dict__.update(dict)

p1 = Profesor(**{"nombre": "Pablo", "apellidos": "Ramos"})
p2 = Profesor(nombre="Pablo", apellidos="Ramos")

datos_profesores = [{"nombre": "Pablo", "apellido": "Ramos"}, {"nombre": "Ana Isabel", "apellido": "Sánchez"}, {"nombre": "Carlos", "apellido": "Vallez"}]
profesores = []
for p in datos_profesores:
    profesores.append(Profesor(**p))

for p in profesores:
    print(p.__dict__)

{'nombre': 'Pablo', 'apellido': 'Ramos'}
{'nombre': 'Ana Isabel', 'apellido': 'Sánchez'}
{'nombre': 'Carlos', 'apellido': 'Vallez'}
```